

bond_pricing

September 11, 2026

1 Corporate Bond Pricing in OSEM

This notebook walks through how OSEM treats a corporate bond position: from raw input data, to projected cash flows, to a calibrated market-consistent price. Each corporate bond is defined by its characteristic, the market conditions such as spread and price.

On a very high level:

- 1) Each corporate bond is saved into a CorpBond object.
- 2) A portfolio of CorpBond objects is then saved into the CorpBondPortfolio.
- 3) Based on the CorpBondPortfolio and market conditions, the cash flows resulting from each bond are reconstructed into a matrix
- 4) Based on the cash flows and current market conditions, the z-spread is calculated using the bisection algorithm to return the current market price

1.1 Notation

This table defines the mathematical notation that will be used through the rest of the workbook

Symbol	Meaning
MD	modelling date
I	issue date
M	maturity date
N	notional amount
c	coupon rate (per period)
f	coupon frequency (payments/year)
MV	market value (price) at the modelling date
z	bond-specific z-spread
$y(t)$	risk-free spot yield for maturity t , from the calibrated EIOPA curve

1.1.1 Step 1 — Cash flow dates

Every bond produces two kinds of cash flow: coupons and the return of notional at maturity.

Notional repayment date:

$$t_M^d = M$$

Coupon dates:

$$I < t_1^d < t_2^d < \dots < t_k^d \leq M$$

1.1.2 Step 2 — Date fractions relative to the modelling date

Only cash flows after MD matter. Each remaining cash flow date is converted to a year fraction from the modelling date:

$$t_i = \frac{t_i^d - MD}{365.25}$$

This is what lets the discounting step treat every cash flow generically as “an amount, this many years from now.”

1.1.3 Step 3 — Cash flow amounts

Notional cash flow (paid once, at maturity):

$$cf_M = N$$

Coupon cash flows (paid at each coupon date):

$$cf_i = N \cdot c$$

1.1.4 Step 4 — Pricing off the risk-free curve

Before any bond-specific adjustment, each cash flow can be discounted purely with the risk-free spot curve calibrated in `CurvesClass`:

$$MV_{\text{rf}} = \sum_{i=1}^k \frac{cf_i}{(1 + y(t_i))^{t_i}} + \frac{cf_M}{(1 + y(t_M))^{t_M}}$$

This is *not* the bond’s actual market price — it ignores credit/liquidity risk entirely. Compare it below to the real `Market_Price` from the input file: the gap is exactly what the z-spread in Step 5 is calibrated to close.

1.1.5 Step 5 — Calibrating the z-spread to match the market price

OSEM finds a single constant z added to the risk-free yield at every maturity such that the discounted cash flows reproduce the bond’s observed market price exactly:

$$MV = \sum_{i=1}^k \frac{cf_i}{(1 + y(t_i) + z)^{t_i}} + \frac{cf_M}{(1 + y(t_M) + z)^{t_M}}$$

There is no closed-form solution for z , so OSEM finds it numerically with a **bisection search** (`bisection_spread`): it repeatedly halves the interval $[z_{\text{start}}, z_{\text{end}}]$ until the priced value is within `precision` of MV .

1.1.6 Step 6 — Calibrating the whole portfolio

`calibrate_bond_portfolio` simply repeats Step 5’s bisection for every bond in the portfolio, storing one z per asset in `zspread_df`.### Step 7 — How this is used in the full OSEM run

This notebook prices bonds once, at $t = 0$. In a full run (`main.py`):

1. The portfolio’s z -spreads are calibrated **once**, at the modelling date (`main.py:255`).
2. In every subsequent period of the simulation, `price_bond_portfolio` is called again with that *same, fixed* z -spread but the *period’s* point on the projected yield curve (`main.py:312`) — so the bond’s price moves only because the risk-free curve and remaining cash flows change, not because its credit spread is re-estimated.

This notebook only demonstrates the fixed-income leg in isolation; the full run also handles equities, cash, liabilities/unit-linked policies, and portfolio rebalancing each period.

1.1.7 Step 7 — How this is used in the full OSEM run

This notebook prices bonds once, at $t = 0$. In a full run (`main.py`):

1. The portfolio’s z -spreads are calibrated **once**, at the modelling date (`main.py:255`).
2. In every subsequent period of the simulation, `price_bond_portfolio` is called again with that *same, fixed* z -spread but the *period’s* point on the projected yield curve (`main.py:312`) — so the bond’s price moves only because the risk-free curve and remaining cash flows change, not because its credit spread is re-estimated.

This notebook only demonstrates the fixed-income leg in isolation; the full run also handles equities, cash, liabilities/unit-linked policies, and portfolio rebalancing each period.

1.1.8 Generating new bonds (not yet implemented)

OSEM’s documentation also describes a process for generating *new* corporate bonds at each future period (to replace maturing debt), calibrating their coupon so they price at par:

$$1 = \sum_{i=1}^k \frac{dy}{(1 + y(t_i) + c + s + ss)^{t_i}} + \frac{1}{(1 + y(t_M) + c + s + ss)^{t_M}}$$

This is not yet implemented in `BondClasses.py` / this notebook — see `Documentation/OSEM_Documentation_draft.ipynb` (cells 84–99) for the full methodology.

1.2 Preparation before running the notebook

1.2.1 Importing necessary external packages

```
[168]: import os
import sys
import numpy as np
import pandas as pd
import datetime as dt
from pathlib import Path
```

1.2.2 Import functions from the main code

```
[169]: from CurvesClass import Curves
from ImportData import import_SWEiopa, get_corporate_bonds, get_configuration,
↳ get_settings
from BondClasses import *
from ConfigurationClass import Configuration
from MainLoop import create_cashflow_dataframe
```

1.2.3 Oriendt base folder

```
[170]: base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
↳ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder))
os.chdir(base_folder)
base_folder = os.getcwd() # Get current working directory
```

```
[171]: conf: Configuration
conf = get_configuration(os.path.join(base_folder, "ALM.ini"), os)
```

These lines of code just extract the absolute location of different files:

```
[172]: parameters_file = conf.input_parameters
cash_portfolio_file = conf.input_cash_portfolio
bond_portfolio_file = conf.input_bond_portfolio
```

```
[173]: paramfile = pd.read_csv("Input/Parameters.csv").set_index("Parameter")
```

The parameter file is:

```
[174]: display(paramfile)
```

	Value
Parameter	
EIOPA_param_file	Input/Param_no_VA.csv
EIOPA_curves_file	Input/Curves_no_VA.csv
country	Slovenia
run_type	Risk Neutral
n_proj_years	50
Precision	1E-10
Tau	0.0001
compounding	-1
Modelling_Date	29/04/2023
liability_mode	unit_linked
random_seed	42

The settings object holds data about file locations, information about the run settings and model parameters such as modelling date.

```
[175]: settings = get_settings(parameters_file)
```

1.3 Corporate bonds

The CorpBond object contains information about each equity position. This includes: * asset_id * nace * issuer * issue_date * maturity_date * coupon_rate * Comprehensive bond spread * notional_amount * frequency * recovery_rate * default_probability * units * market_price

A Python generator reads the bond portfolio file and encodes it into a dictionary based on the asset id. Each asset id contains a CorpBond object describing a single fixed income position.

```
[176]: bond_input_generator = get_corporate_bonds(bond_portfolio_file)
bond_input = {corp_bond.asset_id: corp_bond for corp_bond in
↳bond_input_generator}
```

As an example, a single corporate bond in a CorpBond would look something like:

```
[177]: display(bond_input[1234])

CorpBond(asset_id=1234, nace='A1.4.5', issuer=None, issue_date=datetime.
↳date(2021, 12, 3), maturity_date=datetime.date(2026, 12, 12), coupon_rate=0.
↳03, notional_amount=100.0, spread_country=0.0, spread_sector=0.0, zspread=0.
↳01, spread_stress=0.0, frequency=1, recovery_rate=0.4, default_probability=0.
↳03, units=100.0, market_price=94.0)
```

The CorpBondPortfolio is just a dictionary of such objects.

```
[178]: bond_portfolio = CorpBondPortfolio(bond_input)
```

1.4 Projection of cash flows

Importing the information about the economic environment import_SWEiopa() reads the necessary data about the current yield curve. One of these parameters (the ufr or ultimate forward rate) is necessary in the equity example as ufr is used in the Gordon growth formula to calculate the terminal value of the equity position. Inside OSEM, the parameters related to the yield curve are saved in the Curves object.

```
[179]: [maturities_country, curve_country, extra_param, Qb] = import_SWEiopa(settings.
↳EIOPA_param_file,
↳settings.EIOPA_curves_file, settings.country)
# Curves object with information about term structure
curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau,
↳settings.modelling_date,
settings.country)
```

```
[180]: ufr = extra_param["UFR"]/100 # ultimate forward rate
precision = float(settings.precision) # Numeric precision of the optimisation
# Targeted distance between the extrapolated curve and the ufr at the
↳convergence point
tau = float(settings.tau) # 1 basis point
```

```
[181]: curves.SetObservedTermStructure(maturity_vec=curve_country.index.tolist(),  
    ↪yield_vec=curve_country.values)  
curves.CalcFwdRates()  
curves.ProjectForwardRate(settings.n_proj_years)  
curves.CalibrateProjected(settings.n_proj_years, 0.05, 0.5, 1000)
```

1.4.1 Calibrating the z-spread to match the market price

```
[182]: spreadfile = pd.read_csv("Input/Sector_Spread.csv")  
spreadfile.index = spreadfile["NACE"]  
del spreadfile["NACE"]
```

1.4.2 Cash flow projection of a bond portfolio

The basis of OSEM is cash flow simulation. The cash flows for the coupon payment and the return of the notional are simulated separately.

A list of dictionaries containing all the dates and amounts of coupon payments are produced by calling the `create_coupon_flows` function:

Save the calibration parameters of the selected curve into the Curves instance:

```
[183]: dividend_flows = bond_portfolio.create_coupon_flows(settings.modelling_date,  
    ↪settings.end_date)
```

The list of dictionaries containing the return of the notional amount is produced by calling the function `create_maturity_flows`:

```
[184]: terminal_flows = bond_portfolio.create_maturity_flows(terminal_date=settings.  
    ↪end_date)
```

All cash flows can be represented in a matrix with all possible cash flow dates as columns and all equities as rows. The non-zero entries then represent the value of the cash flow at that date. The first step is to calculate the unique dates for the entire portfolio of bonds. This is done by calling the `unique_dates_profiles()` function over the dates related to coupons or notional amount payments.

Both can then conveniently be represented as DataFrames.

Note that a vector of bond specific spreads is also provided as output.

```
[185]: unique_list = bond_portfolio.unique_dates_profile(dividend_flows)
```

```
[186]: unique_terminal_list = bond_portfolio.unique_dates_profile(terminal_flows)
```

Using the sorted list of unique dates as column headers, the dataframes containing the information related to the cash flows can be produced.

The first dataframe contains the market price of each position. Additionally, the dataframe of zspreads is returned that helps to price the bonds using a discounted cash flow method. The last output is a dataframe containing the amount (units) of each bond in the portfolio is created.

```
[187]: [market_price_df, zspread_df, units_df] = bond_portfolio.  
        ↪init_bond_portfolio_to_dataframe(settings.modelling_date)
```

A dataframe of cash flows and notional amount payments is created:

```
[188]: # Dataframe with bond coupon cash flows  
cash_flows = create_cashflow_dataframe(dividend_flows, unique_list)  
# Dataframe with bond notional cash flows  
notional_cash_flows = create_cashflow_dataframe(terminal_flows, ↪  
        ↪unique_terminal_list)
```

Cash flow dataframe with coupon amounts and dates:

```
[189]: cash_flows.head()
```

```
[189]:
```

	2023-05-03	2023-06-03	2023-06-30	2023-07-03	2023-08-03	2023-09-03	\
1234	0.0	0.0	0.0	0.0	0.0	0.0	
2889	0.0	5.0	0.0	0.0	0.0	0.0	
31	4.0	4.0	0.0	4.0	4.0	4.0	
1	0.0	0.0	1.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	0.0	

	2023-10-03	2023-11-03	2023-12-03	2023-12-30	...	2029-06-30	\
1234	0.0	0.0	3.0	0.0	...	0.0	
2889	0.0	0.0	5.0	0.0	...	0.0	
31	4.0	4.0	4.0	0.0	...	0.0	
1	0.0	0.0	0.0	1.0	...	1.0	
2	0.0	0.0	0.0	0.0	...	0.0	

	2029-12-30	2029-12-31	2030-06-30	2030-12-31	2031-12-31	2032-12-31	\
1234	0.0	0.0	0.0	0.0	0.0	0.0	
2889	0.0	0.0	0.0	0.0	0.0	0.0	
31	0.0	0.0	0.0	0.0	0.0	0.0	
1	1.0	0.0	1.0	0.0	0.0	0.0	
2	0.0	1.0	0.0	1.0	1.0	1.0	

	2033-12-31	2034-12-31	2035-12-31
1234	0.0	0.0	0.0
2889	0.0	0.0	0.0
31	0.0	0.0	0.0
1	0.0	0.0	0.0
2	1.0	1.0	1.0

[5 rows x 66 columns]

Cash flow dataframe with notional amount payments and dates:

```
[190]: notional_cash_flows.head()
```

```
[190]:
```

	2025-12-03	2026-12-12	2028-12-12	2030-06-30	2035-12-31
1234	0.0	100.0	0.0	0.0	0.0
2889	0.0	0.0	100.0	0.0	0.0
31	100.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	100.0	0.0
2	0.0	0.0	0.0	0.0	100.0

The extra spread due to the extra riskiness of the bond compared to a risk free instrument:

```
[191]: display(zspread_df)
```

	2023-04-29
1234	0.010
2889	0.010
31	0.010
1	0.005
2	0.005
3	0.010
4	0.010
5	0.010
6	0.010
7	0.010
8	0.010
9	0.010
10	0.010
11	0.010
12	0.010
13	0.010
14	0.010
15	0.010
16	0.010
17	0.010
18	0.010
19	0.010
20	0.010
21	0.010
22	0.010
23	0.010
24	0.010
25	0.010
26	0.010
27	0.010
28	0.010
29	0.010
30	0.005

1.4.3 Calculation of present value of each instrument

The cashflows can be used to price the current market value of the bond, implied by the assumed economic parameters.

This pricing is done using the risk free rate as the discounting factor. In practice, the price of risk for an equity share is positive.

A calibration method needs to be used to calculate the spread implied by the market. This example will show the pricing using the risk free rate assumptions and the calibration that returns the spread such that the observed market price is preserved.

For simplicity, this example does the pricing at the modelling date by setting the projection year equal to 0.

```
[192]: proj_period = 0
```

The present value of the bond implied by the current yield structure is:

```
[193]: market_price_df = bond_portfolio.price_bond_portfolio(cash_flows,
    ↳ notional_cash_flows, settings, proj_period, curves, zspread_df,
    ↳ market_price_df, settings.modelling_date)
```

```
C:\Users\grego\anaconda3\Lib\site-packages\numpy\core\fromnumeric.py:86:
FutureWarning: The behavior of DataFrame.sum with axis=None is deprecated, in a
future version this will reduce over both axes and return a scalar. To retain
the old behavior, pass axis=0 (or do not pass axis)
    return reduction(axis=axis, out=out, **passkwargs)
C:\Users\grego\OneDrive\Documenti\GitHub\Open_Source_Economic_Model\BondClasses.
py:242: FutureWarning: Calling float on a single element Series is deprecated
and will raise a TypeError in the future. Use float(ser.iloc[0]) instead
    disc_value: float = float(np.sum(nodisc_value))
```

```
[194]: display(market_price_df)
```

```
      2023-04-29
1234    96.375799
2889   131.437251
31     210.297223
1      88.803261
2      71.347086
3      96.375799
4     131.437251
5     210.297223
6      96.375799
7     131.437251
8     210.297223
9      96.375799
10    131.437251
11    210.297223
12     96.375799
```

```

13    131.437251
14    210.297223
15     96.375799
16    131.437251
17    210.297223
18     96.375799
19    131.437251
20    210.297223
21     96.375799
22    131.437251
23    210.297223
24     96.375799
25    131.437251
26    210.297223
27     96.375799
28    131.437251
29    210.297223
30     88.803261

```

1.4.4 Calibrate the spread to match market price

To calibrate the spread implied by the market, OSEM uses a bisection method to obtain the spread such that when added on top of the risk free term structure, the discounted cashflows equal to the current market price.

```

[195]: calibrated_spread = bond_portfolio.corporate_bonds[1234].
      ↪ bisection_spread(x_start=-0.2
                        , x_end=0.2
                        , modelling_date=settings.modelling_date
                        , end_date=settings.end_date
                        , proj_period=proj_period
                        , curves=curves
                        , precision= 0.00000001
                        , max_iter=100000)

```

```

C:\Users\grego\anaconda3\Lib\site-packages\numpy\core\fromnumeric.py:86:
FutureWarning: The behavior of DataFrame.sum with axis=None is deprecated, in a
future version this will reduce over both axes and return a scalar. To retain
the old behavior, pass axis=0 (or do not pass axis)
    return reduction(axis=axis, out=out, **passkwargs)
C:\Users\grego\OneDrive\Documenti\GitHub\Open_Source_Economic_Model\BondClasses.
py:242: FutureWarning: Calling float on a single element Series is deprecated
and will raise a TypeError in the future. Use float(ser.iloc[0]) instead
    disc_value: float = float(np.sum(nodisc_value))

```

The market value calculated using the discounted cash flow method using the calibrated zspread is:

```
[196]: calibrated_bodn = bond_portfolio.corporate_bonds[1234].price_bond(cash_flows.  
    ↪loc[1234],notional_cash_flows.loc[1234],settings.modelling_date,↪  
    ↪proj_period,curves,calibrated_spread)
```

```
[197]: print(calibrated_bodn)
```

94.00000004145151

The function to calibrate the entire portfolio:

```
[198]: zspread_df=bond_portfolio.calibrate_bond_portfolio(zspread_df, settings,↪  
    ↪proj_period, curves)
```

```
[199]: display(zspread_df)
```

	2023-04-29
1234	0.017604
2889	0.100214
31	0.200000
1	0.002904
2	-0.010317
3	0.017604
4	0.100214
5	0.200000
6	0.017604
7	0.100214
8	0.200000
9	0.017604
10	0.100214
11	0.200000
12	0.017604
13	0.100214
14	0.200000
15	0.017604
16	0.100214
17	0.200000
18	0.017604
19	0.100214
20	0.200000
21	0.017604
22	0.100214
23	0.200000
24	0.017604
25	0.100214
26	0.200000
27	0.017604
28	0.100214
29	0.200000
30	0.002904

1.4.5 Appendix input data scheme

There are multiple input files needed to calibrate the fixed income portfolio. They are located in the “Input” folder.

1.4.6 Parameters.csv

Parameters file holds information about the type of run and the modelling date.

- EIOPA_param_file ...the relative location of the EIOPA parameter file that will be used as the RFR Ex. “Input/Param_no_VA.csv”
- EIOPA_curves_file ... the relative location of the EIOPA yield curve that will be used as the RFR Ex. “Input/Curves_no_VA.csv”
- country ... the name of the country that will be used as the base for this run Ex. “Slovenia”
- n_proj_years ... length of a run in years starting from the Modelling date Ex. 50
- Precision ... precision parameter specifying the acceptable tolerance between the calibrated bond price and the market value Ex. 0.00000001
- Tau ... the acceptable size of the gap between the extrapolated yield rate and the ultimate forward rate Ex. 0.0001
- compounding ... the way that the interest rates are compounded in the run Ex. -1
- Modelling_Date ... the starting date of the run specified as a date string Ex.”29/04/2023”

1.4.7 EIOPA RFR files

There are two types of files derived from the monthly EIOPA RFR submission that are used in this model. The “Curves_XX.csv” containing the yearly yield curves for all countries in scope and the “Param_XX.csv” with the parameters used to derive the curves. These files are used to derive the risk free term structure at the modelling date and to efficiently project the evolution of the term structure.

1.4.8 Portfolio description

The modelled portfolio is split by asset classes. The fixed income portfolio is located in the file “Bond_Portfolio.csv”. Each security needs the following fields:

- Asset ID ... unique id such as an ISIN, SEDOL or CUSIP code Ex. IT1234567891
- Asset_Type ... asset type string Ex. “Corporate_Bond”
- NACE ... NACE asset classification code (nomenclature statistique des activités économiques dans la Communauté européenne) Ex. A1.4.5
- Issue_Date ... the string date specifying the issue date of the bond Ex. 3/12/2021
- Maturity_Date ... the string date specifying the maturity date of the bond Ex. 3/12/2021
- Notional_amount ... the notional amount of the bond Ex. 100
- Coupon_Rate ... percentage of the notional amount paid in dividends every period (specified by Frequency) Ex. 0.0014
- Frequency ... number of times per a year that dividends are paid Ex. 1 (once per a year)
- Recovery_Rate ... percentage of the notional amount that can be recovered in case of a default Ex. 0.80
- Default_Probability ... percentage probability of default per year Ex. 0.012
- Units ... number of each bond held in the portfolio Ex. 230
- Market_Price ... market price of the bond at the modelling date Ex. 96

1.4.9 Sector spread

The list of NACE sector codes and the sector specific spread over the risk free rate - NACE ... NACE code of the issuer Ex. "A1.1" - NACE code text ... description of the NACE code for this issuer Ex. "Growing of non-perennial crops" - cSpread ... Country of issuance or operations specific spread Ex. 0.01 - sSpread ... Sector specific spread Ex. 0.01 - zSpread ... bond issuance sector specific spread Ex. 0.01 - ssSpread ... Extra spread assumed by the specific stress scenario Ex. 0.01

In the POC, the spread is displayed directly in the bond input file